# CuadernoEstructuraDeDatos.pdf

rochie

Estructuras de Datos

1º Grado en Matemáticas y Computación

Escuela Politécnica Superior
Universidad de Alcalá

Accede al documento original

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# Estructura de datos

## Grado en Matemáticas y Computación

Cuaderno

**Integrantes**

raquel.vegas@edu.uah.es

sergio.ruizj@edu.uah.es

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# ÍNDICE


1. Abstracción
   1.1. Definición y nociones básicas
   1.2. Tipos abstractos de datos
   1.3. Programación y especificación de los TAD's
2. Programación Orientada a Objetos
   2.1. Polimorfismo
   2.2. Ejemplo
3. Tipos de estructura de datos
   3.1. Estructura de dato estática
   3.2. Estructura de dato dinámica

**ESTRUCTURA DE DATOS LINEALES**

4. Estructuras de datos básicas
   4.1. Listas simples
   4.2. Listas simplemente enlazadas
   4.3. Listas doblemente enlazadas
   4.4. Pilas
   4.5. Colas
5. Otras estructuras de datos
   5.1. Listas indexadas (o indizadas)
   5.2. Listas ordenadas
   5.3. Listas indexadas (o indizadas)
   5.4. Colas de prioridad

**ESTRUCTURA DE DATOS NO LINEALES**

6. Arboles
   6.1. Definicion
   6.2. Terminologia
   6.3. Tipos de árboles
   6.4. Estructuras
   6.5. Diagrama UML
7. Arboles equilibrados
   7.1. Definicion y nociones basicas
   7.2. Tipos de desequilibrios
   7.3. Implementacion
   7.4. Diagrama UML
8. Grafos
   8.1. Definicion y nociones basicas
   8.2. Caracteristicas
   8.3. Implementación
   8.4. Aplicaciones reales
   8.5. Diagrama UML




WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

## 9. Bibliografía



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# Abstracción

## DEFINICIÓN Y NOCIONES BÁSICAS

El concepto de abstracción en la Programación Orientada a Objetos podría definirse como la acción de aislar mentalmente o considerar por separado las cualidades de un objeto. Es decir, con la abstracción sacamos las cualidades principales sin detenernos en los detalles. La abstracción permite tratar la complejidad, entendida como el exceso en el número de detalles.

El proceso de abstracción comprende dos aspectos que son complementarios:

1. Destacar los detalles relevantes del objeto en estudio.
2. Ignorar los detalles irrelevantes.

## TIPOS ABSTRACTOS DE DATOS

Un tipo abstracto de datos (TAD) es un conjunto de propiedades y de operaciones definidas mediante una especificación que es independiente de cualquier representación. Se suele considerar que un TAD es un tipo de datos construido por el programador para resolver una determinada situación.

## PROGRAMACIÓN Y ESPECIFICACIÓN DE LOS TAD

Para definir un TAD, el programador comienza por definir las operaciones que se pueden realizar con él, es decir, que operaciones son relevantes y útiles para operar con las variables pertenecientes al mismo.

La representación de un Tipo Abstracto de Datos es independiente y, de hecho, puede variar sin que afecte a los programas que utilicen el tipo, ya que la especificación se mantiene. El hecho de que el programador desconozca la representación obliga a definir tanto un constructor que permita asignar valores a una variable de ese tipo como procedimientos y funciones que permitan establecer y obtener los valores del tipo (*getters* & *setters*[1]).

En la especificación de un TAD hay que tener especial cuidado, pues debe poseer cuatro propiedades importantes para su correcta aplicación.

1. Precisión → Que implica decir solo lo imprescindible.
2. Generalidad → Que quiere decir que sea adaptable a diferentes contextos.
3. Legibilidad → Entendida como facilidad de comprensión.
4. No-ambigüedad → Que quiere decir que no dé lugar a distintas interpretaciones.

La <u>especificación formal</u> de un tipo abstracto de datos consta de sintaxis (descripción de operaciones, tipos de parámetros y resultados) y semántica (descripción de "lo que hacen" o significado de cada operación).

[1] Métodos definidos en Metodología de la Programación.



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

En cambio, la especificación algebraica es una técnica formal con objetivo de definir, de forma no ambigua e independiente de cualquier implementación un tipo de datos: valores de este y efecto de cada operación.

## Programación Orientada a Objetos

La programación orientada a objetos (POO, en adelante), es un paradigma de la programación basada en que los programas se pueden organizar como una colección de objetos interconectados, cada uno con su conjunto de datos y funcionalidades. Con la POO, lo que buscamos es dejar de centrarnos en la lógica pura de los programas, para empezar a pensar en objetos. Esto nos ayudará a la hora de manejar sistemas mucha más grandes, ya que, en vez de pensar en funciones, pensamos en la relaciones o interacciones de los diferentes componentes del sistema.

La POO hace que el código sea reutilizable, organizado y fácil de mantener. También evita que la gente acceda a datos no deseados o vea nuestro código, gracias a la encapsulación y abstracción.

**POLIMORFISMO**

En la programación orientada a objetos, el polimorfismo es cuando los objetos de una clase pueden comportarse de manera diferente según los parámetros que se utilicen al llamarlos.

- **TIPOS DE POLIMORFISMO**

  Por norma se establece que existen tres tipos de polimorfismo:

  - **Sobrecarga:** El más usado de todos y aplicado cuando se especifican funciones llamadas igual en clases independientes la una de la otra.
  - **Paramétrico:** Existencia de funciones con el mismo nombre pero que necesitan argumentos de distinto tipo.
  - **Inclusión:** Cuando se puede llamar a un método sin conocer su tipo, por lo que no se tienen en cuenta los detalles de las clases especializadas, usando una interfaz común.

- **EJEMPLO**

  Para dar un ejemplo sencillo de polimorfismo crearemos dos clases distintas: Gato y Perro, que heredan de la superclase Animal. La clase Animal alberga un método abstracto "*makeSound()*", implementado de distinta forma en cada una de las subclases.

```
class Animal {
    public void makeSound() {
        System.out.println("Grr...");
    }
}
class Cat extends Animal {
    public void makeSound() {
        System.out.println("Meow");
    }
```



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}
class Dog extends Animal {
    public void makeSound() {
        System.out.println("Woof");
```

## Tipos de estructura de datos

Las estructuras de datos en programación son herramientas que permiten organizar y representar la información de forma eficiente. Desde *arrays* hasta *árboles* y *grafos*. Estas estructuras nos ayudan a diseñar soluciones para problemas informáticos. La primera división que encontramos dentro de este tema son las estructuras de datos estáticas y las dinámicas.

**ESTRUCTURA DE DATOS ESTÁTICA**

Las estructuras de datos estáticas tienen un tamaño definido antes de la ejecución y no pueden modificarse. Un ejemplo sencillo de esto son los *arrays*.

**ESTRUCTURA DE DATOS DINÁMICA**

Una estructura de datos dinámica puede cambiar de tamaño durante la ejecución del programa. Pueden ser ordenados linealmente o con estructuras algo más complejas.

- **ESTRUCTURA DE DATOS LINEALES**

  Las estructuras de datos lineales son aquellas en las que los elementos están ubicados uno tras otro, cada uno con un único predecesor y sucesor. Hay tres tipos:

  - Listas enlazadas
  - Pilas
  - Colas

- **ESTRUCTURA DE DATOS NO LINEALES**

  Las estructuras de datos no lineales son aquellas en las que cada elemento puede estar enlazado con cualquier otro, teniendo uno o varios sucesores y predecesores. Hay dos tipos:

  - Árboles
  - Grafos



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# ESTRUCTURA DE DATOS

## *Lineales*

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

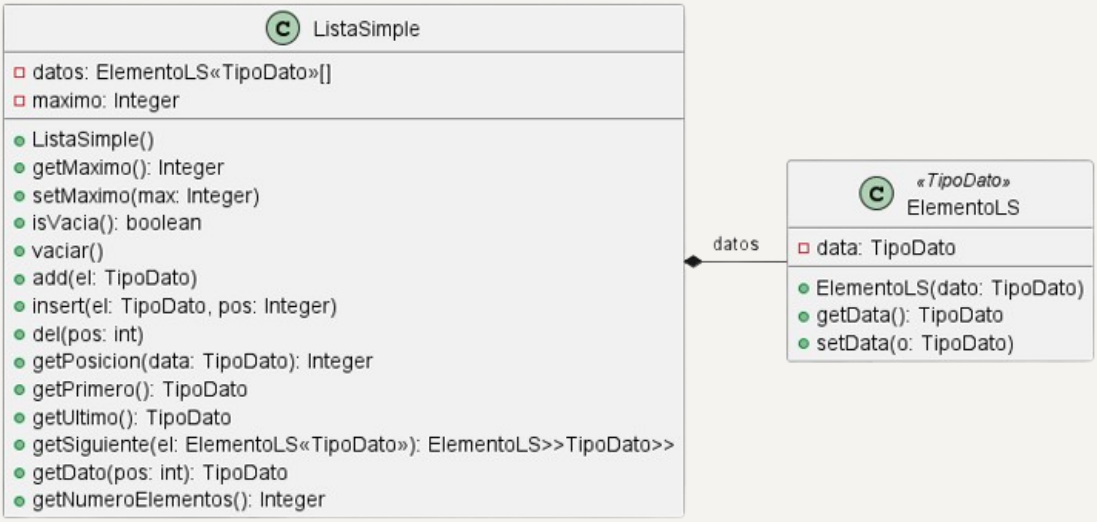
En esta sección se desarrollarán distintas estructuras de datos lineales, ya que los componentes ocupan lugares sucesivos en la estructura y cada uno tiene un único suceso y un predecesor, excepto el último y el primero.

**LISTAS SIMPLES**

Una lista simple es una estructura de datos que nos permite almacenar una ristra de datos de un mismo tipo. El tamaño de estas debe ser declarado en un primer momento y no puede cambiar en tiempo de ejecución, a no ser que se defina un método específico para ello.

En este tipo de listas los elementos están numerados con una numeración que comienza desde el elemento 0 hasta el elemento n-1. Es decir, una lista con 10 elementos irá del 0 al 9.

Como ya hemos visto en la parte de prácticas de la asignatura, el diagrama UML (Unified Modeling Lenguage) asociado a esta estructura es el siguiente.




**LISTAS SIMPLEMENTE ENLAZADAS**

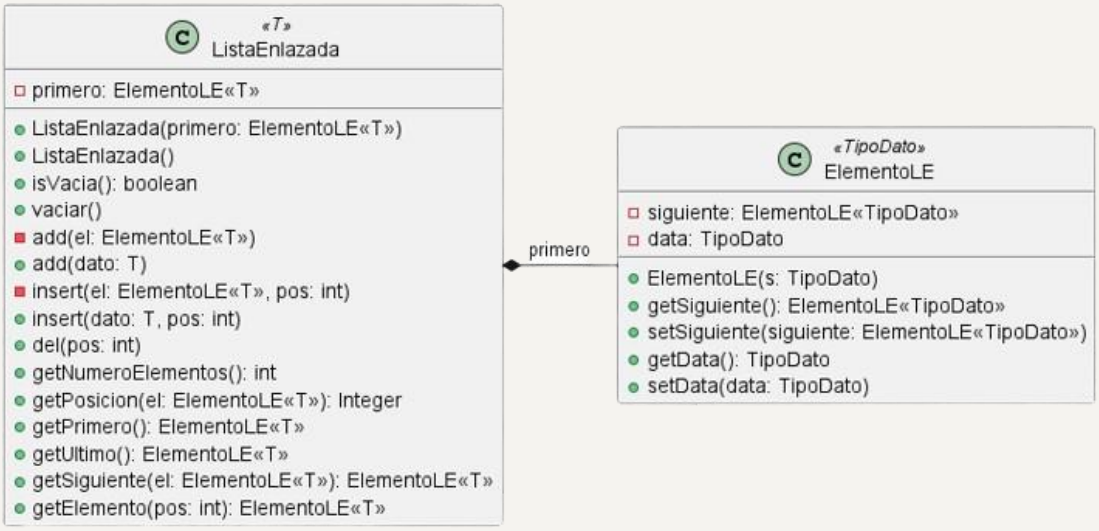
Una lista enlazada simple es una estructura de datos en la que cada elemento apunta al siguiente. En este tipo de estructura los elementos no están numerados, no obstante, teniendo la referencia del principio de la lista podemos acceder a todos los elementos de esta.

Es importante saber que, en este tipo de listas, cuando un elemento no tiene ninguno que le esté apuntando, este se comporta como inexistente. Por lo que para la definición de los métodos de añadir o eliminar elementos, solo hay que jugar con los enlaces de estos.

Como ya hemos visto en la parte práctica de la asignatura, el diagrama UML asociado a esta estructura es el siguiente.



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

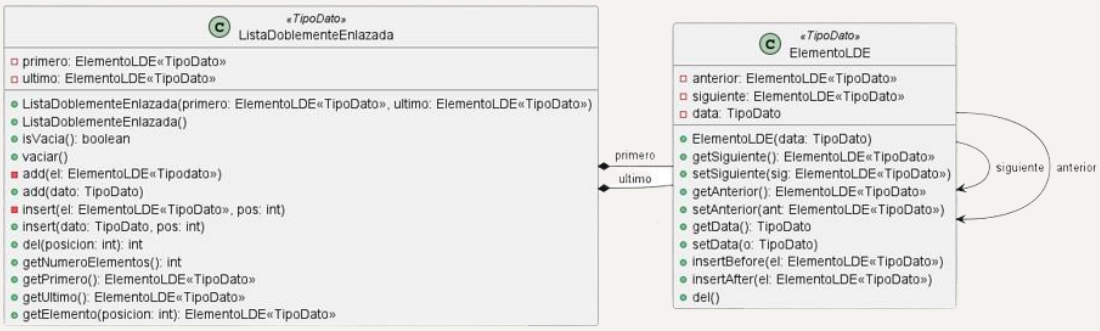



## LISTAS DOBLEMENTE ENLAZADAS

Una lista doblemente enlazada es una estructura de datos en la que cada elemento apunta al anterior y al siguiente. De este modo, se puede recorrer la lista de un lado al otro y viceversa, proporcionando una ventaja frente a las listas simplemente enlazadas.

Las listas enlazadas son una estructura de datos muy recurrida para su implementación en diversas aplicaciones. Algunos ejemplos de ellas son el manejo de memoria o la implementación de otras estructuras de datos más complejas como pilas o colas.

Como ya hemos visto en el apartado práctico de la asignatura, el diagrama UML asociado a esta estructura de datos es el siguiente.




## PILAS

Estructura de datos en la que solo se pueden añadir y extraer elementos desde uno de los lados de la lista (que denominaremos "final"). Estas siguen el método LIFO[2], que denota que el último elemento en entrar a la estructura será el primero que tenga que salir. Una buena similitud para entender el concepto es imaginarse una pila de libros exhibida en una librería, en la que solo podrás seguir poniendo libros encima del resto, o quitar los libros uno a uno desde arriba.

[2] Last In, First Out

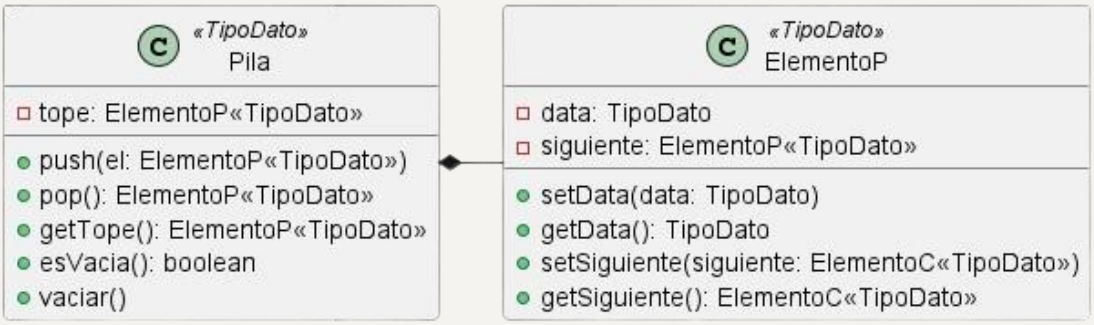


Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

La administración de este tipo de estructuras es muy sencilla ya que tiene pocas operaciones. Sin embargo, en caso de querer acceder a un elemento que no se encuentre en el final de la cola se debe realizar un volcado de la misma a una pila auxiliar. Algunas de las operaciones básicas definidas en las pilas pueden ser:

- Push(): Añade un valor al final de la pila.
- Pop(): Retira un valor del final de la pila.
- Cima(): Devuelve el valor en la posición final de la pila.
- esVacia(): Devuelve si la pila está vacía o no
- Vaciar(): Elimina todos los elementos de la pila.

El diagrama UML asociado a esta estructura de datos sería así.




- **EJEMPLO: TORRES DE HANOI**

Un ejemplo del el uso de las pilas es con el juego matemático de la Torre de Hanoi. Esta se basa en tres postes fijados a un tablero a los que se apilan discos perforados de radio creciente insertándolos uno encima de otro.

Para programar este juego hay que fijarnos en el algoritmo que hay detrás de este, pero una vez entendido este algoritmo es fácil usar las pilas para programar nuestro propio juego de Hanoi.

Las pilas van a ser los tres postes por lo que los definiremos de esta manera:

```
private Pila torreOrigen;
5 usages
private Pila torreAuxiliar;
5 usages
private Pila torreDestino;
```

Tras esto creamos el constructor y empezamos a programar el método que iniciará el juego.

```
public void iniciarJuego(int numAros) {
    for (int i = numAros; i > 0; i--) {
        torreOrigen.push(i);
    }
    imprimirEstadoTorres();
    resolverHanoi(numAros, torreOrigen, torreDestino, torreAuxiliar);
}
```



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Con este método decimos cuántos aros queremos tener en nuestro juego y los vamos almacenando en la pila llamada *torreOrigen* de mayor tamaño a menor (El tamaño viene definido por el número de aro que sea). Tras haber almacenado todos los aros llamamos a la función que imprimirá el estado de todas las torres:

```
2 usages   new *
private void imprimirEstadoTorres() {
    System.out.println("Torre Origen: " + torreOrigen.toString());
    System.out.println("Torre Auxiliar: " + torreAuxiliar.toString());
    System.out.println("Torre Destino: " + torreDestino.toString());
    System.out.println("--------------");
}
```

Tras esto llamamos a la función que resolverá nuestra Torre de Hanoi:

```
private void resolverHanoi(int n, Pila origen, Pila destino, Pila auxiliar) {
    if (n > 0) {
        resolverHanoi( n: n - 1, origen, auxiliar, destino);
        moverDisco(origen, destino);
        imprimirEstadoTorres();
        resolverHanoi( n: n - 1, auxiliar, destino, origen);
    }
}
```

Este código sigue el algoritmo que tiene por detrás este juego, moviendo cada disco a una pila determinada con la siguiente función:

```
private void moverDisco(Pila origen, Pila destino) {
    String origenstr = null;
    String destinostr = null;
    if (origen==torreOrigen){
        origenstr = "torreOrigen";
    } else if (origen == torreAuxiliar) {
        origenstr = "torreAuxiliar";
    } else if (origen == torreDestino) {
        origenstr = "torreDestino";
    }
    if (destino==torreOrigen){
        destinostr = "torreOrigen";
    } else if (destino == torreAuxiliar) {
        destinostr = "torreAuxiliar";
    } else if (destino == torreDestino) {
        destinostr = "torreDestino";
    }
    Integer disco = origen.pop();
    destino.push(disco);
    System.out.println("Mover disco " + disco + " de " + origenstr + " a " + destinostr);
```

Y con todo ya programado podríamos crear ya el main para ejecutar nuestro juego:

```
public class MainTorreHanoi {
    new *
    public static void main(String[] args) {
        System.out.println("--------------");
        TorreHanoi juego = new TorreHanoi();
        juego.iniciarJuego( numAros: 3);
```



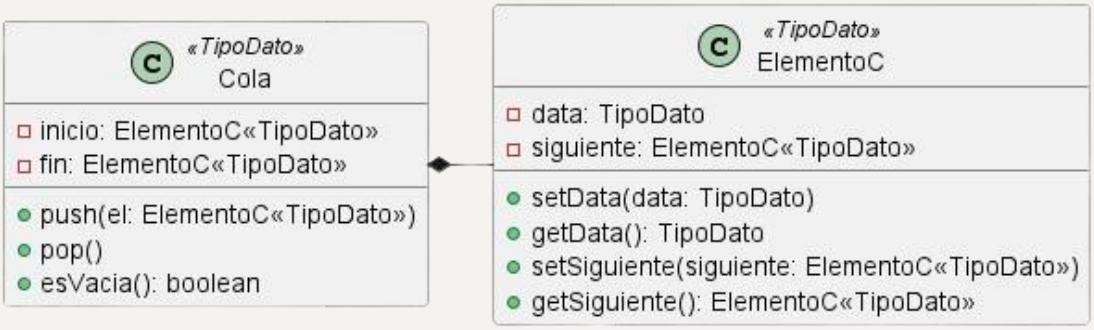
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

**COLAS**

Estructura de datos similar a la pila en la que se añaden elementos por uno de los extremos y se extraen por el otro. Siguiendo el método FIFO[3], que hace referencia a que el primer elemento que se introduzca en la estructura será el primero que salga de ella. Algunas de los métodos implementados son:

- Push(): Insertar un elemento en la cola.
- Pop (): Eliminar un elemento de la cola (desencolar).
- esVacia(): Comprobar si la pila está vacía.

El diagrama UML asociado a esta estructura de datos sería algo así.




## Otras estructuras de datos

Estos tipos de listas se basan en la estructura de las listas doblemente enlazadas, cada elemento está relacionado con su elemento anterior y con el siguiente.

**LISTAS ORDENADAS**

La característica principal de una lista ordenada es que, al insertar un elemento nuevo, esta no lo hace de manera arbitraria, sino que sigue un orden claro e inserta el nuevo elemento en la posición adecuada.

Para poder comparar los elementos y llevar a cabo la inserción de forma ordenada, usamos la interfaz Comparable. Que compara los objetos a añadir y devuelve 1, 0 o -1 en función de su "posición" con respecto a otro elemento ya en la lista. Suelen ser comparaciones de valores numéricos, sin embargo, a veces se llevan a cabo comparaciones de elementos tipo *String*, que se hacen por orden alfabético.

Las listas ordenadas son un tipo de listas poco útiles si se tratan listas con muchos elementos, pues hay que recorrerlos uno a uno para saber dónde insertar elementos nuevos.

**LISTAS INDEXADAS**

Listas que pueden tener elementos "iguales" pero con algún dato identificativo que los distinga. Un buen ejemplo de ello pueden ser los coches, que pueden ser idénticos

[3] First In, First Out



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

(misma marca, modelo, color…), pero siempre serán distintos, pues sus matrículas lo serán.

Este tipo de listas tiene el mismo problema que las listas ordenadas, hay que recorre cada elemento de la lista uno a uno para poder encontrar la posición del elemento a añadir.

**COLAS DE PRIORIDAD**

Una cola de prioridad es un TAD similar a una cola en la que los elementos de añaden además con una prioridad previamente asignada. En esta estructura de datos, un elemento de mayor prioridad será desencolado antes que un elemento con una prioridad menor.

En esta forma de almacenaje de datos existen los mismos métodos que en las colas comunes. Sin embargo, al añadir un elemento hay que insertar el elemento a añadir y la prioridad que tiene. Pues luego al intentar eliminar un elemento, se eliminará el elemento con mayor prioridad.



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# ESTRUCTURA DE DATOS

## *No Lineales*

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# Árboles

## DEFINICIÓN

Los árboles en estructura de datos representan datos con una relación jerárquica, clasificándose en árboles generales y árboles binarios. Formalmente, un árbol es un conjunto finito de elementos con nodos que se dividen en subárboles de la raíz de forma recursiva. Hay dos tipos principales de árboles: generales y binarios.

## TERMINOLOGÍA

- Nodo: elemento de un árbol.
- Enlace: conexión entre dos nodos.
- Nodos raíz: superior en jerarquía.
- Camino: enlace entre dos nodos. (No todos los nodos están conectados por un camino)
- Rama: camino que termina en una hoja.
- Grado de un nodo: número de subárboles que tiene.
- Nivel de un nodo: número de enlaces desde la raíz hasta el nodo.
- Altura de un árbol: número máximo de nodos en una rama más uno.
- Peso de un árbol: número de nodos terminales.

## TIPOS DE ÁRBOLES

- **ÁRBOLES GENERALES**

  Los árboles generales permiten un número ilimitado de subárboles

- **ÁRBOLES BINARIOS**

  Los árboles binarios tienen como máximo dos nodos sucesores por elemento

## ESTRUCTURAS

Los árboles pueden tener diferentes estructuras dependiendo del uso que le queramos dar. Algunas de estas estructuras son las siguientes:

- **DE BÚSQUEDA**

  Estos árboles son siempre árboles binarios. Para estos árboles se sigue unos criterios determinados:

  - El primer elemento de la lista de los datos que se quieran introducir será la raíz.
  - Desde determinar la raíz los siguientes elementos de colocan dependiendo de si son mayores o menores que la raíz. Si son menores se colocan el subárbol izquierdo y los mayores en el derecho.



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- **EQUILIBRADO**

  Las alturas de los dos subárboles de cada nodo difieren en un máximo de una unidad.

- **HOMOGÉNEO**

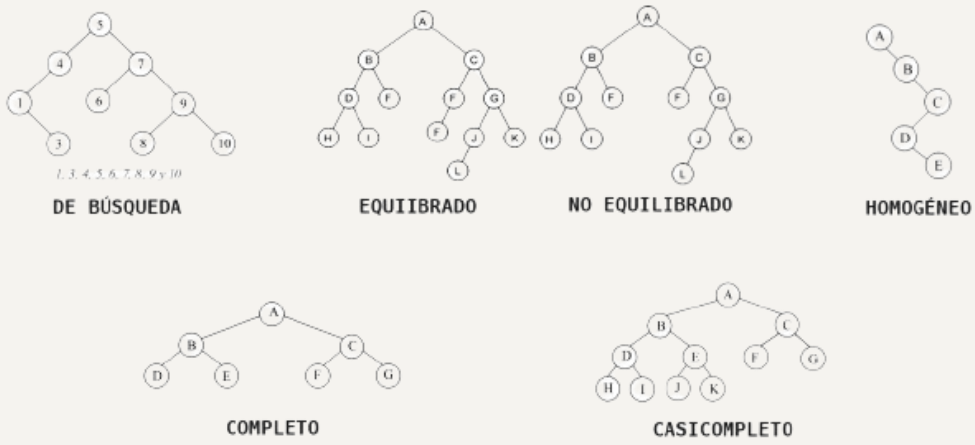
  Todos los nodos tienen el mismo grado.

- **COMPLETO**

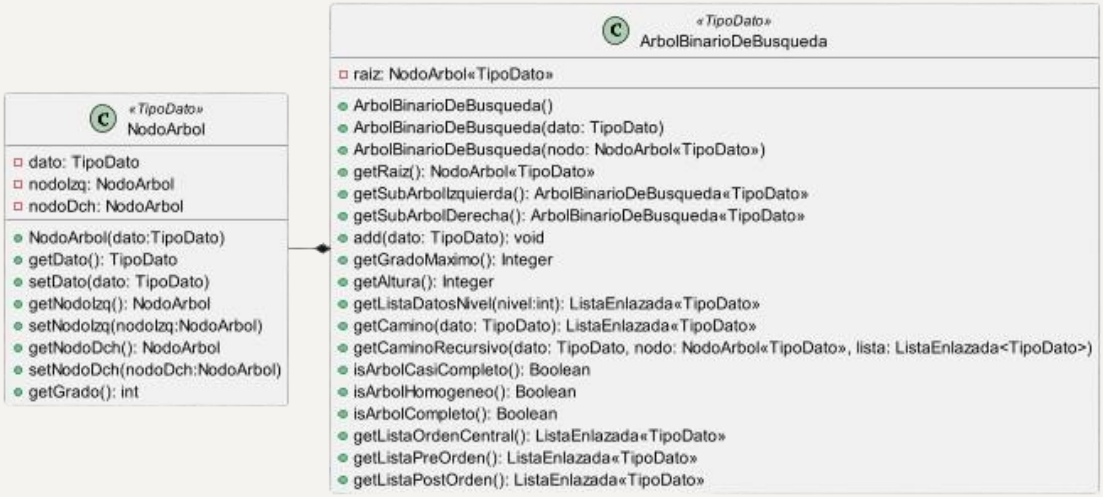
  Todos los nodos a excepción de las hojas tienen grado 2. Además, gracias a esto podemos saber que un árbol binario completo de nivel n tiene 2n -1 nodos.

- **CASICOMPLETO**

  Este tipo de árboles son completos hasta el penúltimo nivel, pero en su último nivel solo tiene nodos por la izquierda, creando una forma de "escalón".




## DIAGRAMA UML ÁRBOL BINARIO



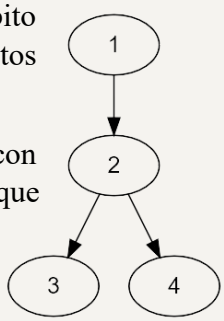



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

## DEFINICION Y NOCIONES BÁSICAS

Durante el transcurso de esta asignatura hemos profundizado en el ámbito de los arboles equilibrados y es conveniente mencionar distintos aspectos acerca de su estructura e implementación.

Los hemos estado usando con la estructura de un árbol binario, es decir, con todos sus nodos con grado máximo 2. Este tipo de árboles, no son más que una subclase de los árboles anteriormente mencionados. La idea de "equilibrados", hace referencia a que la altura de los subárboles de cualquier nodo no difiere en más de una unidad. Por ello, árboles como *árbol 1* son inviables.

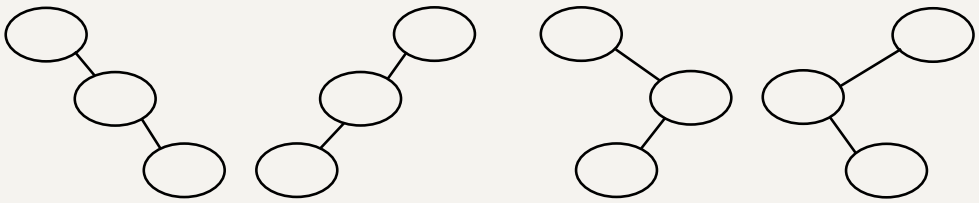


Árbol 1

## TIPOS DE DESEQUILIBRIOS

A la hora de analizar el desequilibrio existente en un nodo podemos diferenciar cuatro casos en los que procederemos de forma distinta para corregir la imperfección. Estos casos, los agrupamos en rotaciones simples, que conllevan una menor complejidad, y rotaciones dobles, que su implementación se basa en la combinación de rotaciones simples.

- Rotaciones simples
  - Rotación derecha-derecha: En este caso, el nodo que posee el desequilibrio tiene un hijo a la derecha y este vuelve a tener otro hacia el mismo lado.
  - Rotación izquierda-izquierda: En cambio, en esta situación el nodo que posee el desequilibrio tiene un hijo a la izquierda que vuelve a tener otro hijo a la izquierda. El proceso de corrección es inverso al del caso anterior.
- Rotaciones dobles:
  - Rotación izquierda-derecha: En este caso, como bien el nombre indica, el nodo "padre" que tiene el desequilibrio posee un hijo a la izquierda, y luego este otro a la derecha. En estos casos, la corrección se basa en aplicar una rotación derecha-derecha sobre el nodo hijo y luego una izquierda-izquierda sobre el padre.
  - Rotación derecha-izquierda: La situación en este caso es contraria al anterior, y se procede también de forma contraria. Se aplicará una rotación izquierda-izquierda sobre el nodo hijo para después aplicar una derecha-derecha sobre el nodo padre.


*Derecha-derecha* *Izquierda-izquierda* *Derecha-izquierda* *Izquierda-derecha*



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

## IMPLEMENTACIÓN

Para su correcta implementación basta con hacer modificaciones en los métodos de adición y eliminación de elementos de los árboles binarios conocidos hasta ahora, que van a ser los únicos que modifiquen su estructura. De forma esquematizada, los cambios se basan en:

- Tras añadir un elemento, como (por norma) el método de añadir se habrá implementado de forma recursiva, lo único que habrá que hacer es recorrer el árbol de abajo (elemento añadido) a arriba (raíz) comprobando si cada uno de los nodos está equilibrando y corrigiendo desequilibrios (en caso de que sea necesario).
- Cuando llamemos al método de eliminación, se seguirá un proceso similar. Al eliminar el elemento pondremos en su posición el siguiente nodo del árbol cuyo dato sea inmediatamente posterior al dato que hemos eliminado. Tras ello, comprobaremos como en la adición de elementos que todos los nodos modificados o implicados en el borrado del elemento están equilibrados y corregiremos desequilibrios en caso de que sea necesario.

## DIAGRAMA UML

En el siguiente diagrama se muestran los métodos básicos para el funcionamiento del árbol. No obstante, se han suprimido algunos métodos privados de las clases usados en al correcto funcionamiento de los métodos públicos que implementan procesos más complejos.

# Grafos

## DEFINICION Y NOCIONES BÁSICAS

Un grafo es una estructura de datos no lineal basada en el almacenamiento de datos y sus relaciones entre ellos. En un grafo, como bien se ha mencionado, vamos a almacenar vértices (nodos, datos), y arcos (que unirán nodos con nodos mostrando su relación).



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

**CARACTERISTICAS**

Dentro de los grafos, se pueden clasificar atendiendo a diversas características de los mismos:

- Grafos ponderados: Grafos en los que cada arco tiene un peso. Un buen ejemplo de ello son los grafos usados para el diseño de mapas virtuales como puede ser "*Google maps"*, donde cada arco entre vértices tiene un peso (su distancia).
- Grafos dirigidos (no dirigidos): Si los arcos tienen dirección, es decir, en vez de conectar dos vértices conectan solo uno con otro, se dice que estamos ante un grafo dirigido. Este tipo de grafos se puede observar en los grafos de conocimiento. Donde los datos se suelen relacionar mediante arcos dirigidos en los que se anota el hecho que los une.

Además, dependiendo de los arcos que acepte el grafo este va a ser simple o el conocido "multigrafo". Si el grafo no acepta dos arcos que unan los mismos vértices ni arcos que unan un vértice consigo mismo, estamos ante un grafo simple. Si, por el contrario, si que se pueden dar estas dos situaciones mencionadas, estamos ante un multigrafo.

**IMPLEMENTACION**

Para la implementación de los grafos hemos determinado que un grafo se compone de una lista de vértices y otra de arcos. Con eso, cada vez que se añaden vértices o arcos se van modificando dichas listas y almacenando en la clase "grafo".

- **ALGORITMO DE DIJKSTRA**

  Uno de los mayores retos que hemos afrontado a la hora de implementar esta estructura de almacenaje fue el hecho de calcular el camino mínimo entre dos vértices. Para ello, recurrimos al famoso algoritmo de Edsger Wybe Dijkstra, reconocido erudito de los Países Bajos. En él, mediante la combinación de las estructuras de datos lineales mencionadas al principio del documento, se va recorriendo el grafo completo hasta al final devolver un mapa con los caminos mínimos de un vértice seleccionado hasta el resto de vértices existentes en el grafo.

**APLICACIONES REALES**

De las estructuras vistas hasta ahora los grafos puede que sean los que juegan un papel más importante en nuestro día a día. Es así por su infinidad de aplicaciones reales.

Una de ellas, y puede que una de las más usada por todos, sea en los grafos de conocimiento. Este tipo de grafos, se implementan en plataformas virtuales como Google, y por ellos se relaciona todo tipo de conocimiento almacenado en Google. Por ejemplo, la persona "Albert Einstein" está relacionado con "Teoría de la relatividad" mediante un arco. De forma que toda la información se almacena relacionándose una con la otra y su búsqueda es mucho más rápida y eficiente.

Como hemos dicho, mediante el uso de un grafo para el almacenaje de información ganamos en velocidad. Tanto es así, que otra aplicación de los grafos es la detección de fraude en tiempo real. Gracias a toda esta tecnología, en cuestión de segundos se ejecutan

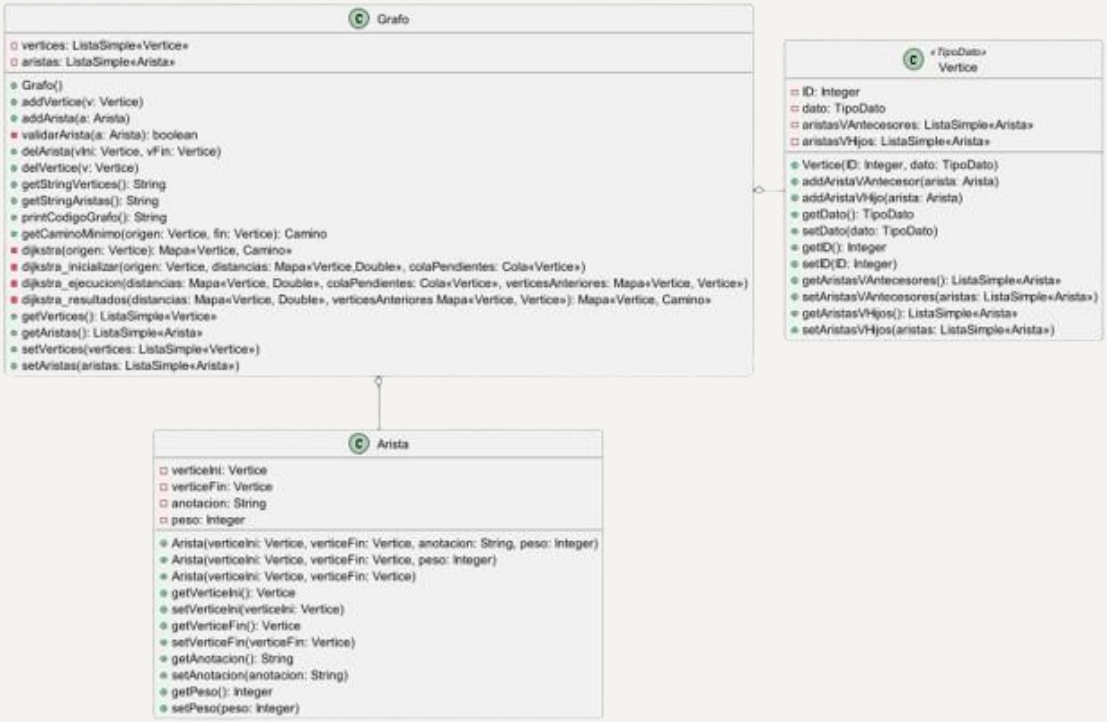


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

cientos de consultas sobre millones de datos por los que se pueden investigar transacciones bancarias a tiempo real.

**DIAGRAMA UML**






Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

# Bibliografía


1. Canelo, Miriam Martínez. "¿Qué Es La Programación Orientada a Objetos?" *Profile Software Services*, 5 Sept. 2023, profile.es/blog/que-es-la-programacion-orientada-a-objetos/#Que_es_la_Programacion_Orientada_a_Objetos.
2. "Polimorfismo En Java: Programación Orientada a Objetos." *IfgeekthenNTTdata*, 25 Aug. 2023, ifgeekthen.nttdata.com/es/polimorfismo-en-java-programaci%C3%B3n-orientada-objetos.
3. *Estructura de Datos En Java*, https://lc.fie.umich.mx/~a1039048f/nts/Documents/Estructura%20de%20datos%20en%20java%20Joyanes%201ed.pdf . Accessed 5 Apr. 2024.
4. *Tema 4. Estructuras No Lineales de Datos: Árboles*, https://www.colimbo.net/documentos/documentacion/113/FPII04_Estructuras_no_lineales_de_datos.pdf. Accessed 5 Apr. 2024.
5. "¿Qué Es Una Estructura de Datos En Programación?" *Henry*, Henry, 10 Feb. 2022, blog.soyhenry.com/que-es-una-estructura-de-datos-en-programacion/#:~:text=Las%20estructuras%20de%20datos%20no,%C3%81rboles.




Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.